UNIVERSIDADE CRUZEIRO DO SUL
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
TÓPICOS AVANÇADOS EM SISTEMAS DE INFORMAÇÃO II

CLAYTON DE ANDRADE JUNIOR – RGM: 45781303
GABRIEL YANES – RGM: 42585953
LEONARDO ROCHA – RGM: 41735285
LUANA APARECIDA SILVA CHE – RGM: 43637302
NICOLAS NERIS DOS SANTOS DOURADO – RGM: 04307588-6
WESLEY RODRIGUES DIAS – RGM: 42288835

PROJETO PRÁTICO AVALIATIVO

API de Gerenciamento de Pedidos
Documentação das rotas, regras de negócio, arquitetura e testes

github.com/Wesleytech22/gerenciamento-pedidos

Projeto prático avaliativo apresentado à disciplina de Tópicos Avançados em Sistemas de Informação II, do curso de Análise e Desenvolvimento de Sistemas.

São Paulo
2026

1 INTRODUÇÃO

Este documento descreve a API REST de gerenciamento de pedidos de uma pequena loja, desenvolvida como projeto prático avaliativo. A aplicação permite cadastrar pedidos, listá-los, consultá-los pelo identificador e atualizar o seu status, aplicando arquitetura em camadas, validação de dados e tratamento centralizado de erros.

O projeto foi dividido em duas aplicações independentes: o **backend**, que expõe a API REST, e o **frontend**, uma interface web que consome essa API. Essa separação permite testar a API diretamente pelo Postman ou pelos testes automatizados, sem depender da interface, e evoluir cada parte de forma isolada.

O código-fonte, as evidências e esta documentação estão disponíveis no repositório: https://github.com/Wesleytech22/gerenciamento-pedidos

Requisitos funcionais atendidos:

Código	Requisito	Rota
RF01	Cadastrar pedido com cliente, tipo, valor e prioridade.	`POST /pedidos`
RF02	Listar os pedidos cadastrados.	`GET /pedidos`
RF03	Consultar um pedido pelo identificador.	`GET /pedidos/:id`
RF04	Atualizar o status de um pedido.	`PATCH /pedidos/:id/status`

2 TECNOLOGIAS UTILIZADAS

Tecnologia	Uso no projeto
Node.js	Ambiente de execução JavaScript do backend e do servidor do frontend.
Express 5	Framework HTTP da API: rotas, leitura de JSON e middlewares.
cors	Libera o acesso da interface web, que roda em outra porta (5173), à API (3000).
Jest + Supertest	Testes automatizados de unidade (service) e de integração (rotas HTTP).
Postman	Testes manuais e geração das evidências.
HTML, CSS e JavaScript	Interface web (frontend), sem frameworks.
Git e GitHub	Versionamento e entrega do código-fonte.

Os dados são armazenados em memória, conforme permitido no enunciado. Por isso, ao reiniciar o backend, os pedidos são apagados e os identificadores voltam a começar em 1.

3 ESTRUTURA DO PROJETO E EXECUÇÃO

```
gerenciamento-pedidos/
├── backend/                    API REST (porta 3000)
│   ├── src/
│   │   ├── server.js           inicia o servidor HTTP
│   │   ├── app.js              configura o Express, CORS, rotas e erros
│   │   ├── routes/             mapeamento das URLs
│   │   ├── controllers/        camada HTTP
│   │   ├── services/           regras de negócio
│   │   ├── validators/         validação dos dados de entrada
│   │   ├── repositories/       armazenamento em memória
│   │   ├── errors/             classes de erro da aplicação
│   │   └── middlewares/        tratamento centralizado de erros
│   ├── tests/                  testes Jest + Supertest
│   └── postman/                coleção de requisições do Postman
├── frontend/                   interface web (porta 5173)
│   ├── server.js               servidor estático sem dependências
│   └── public/                 index.html, styles.css, app.js, config.js
├── evidencias/                 capturas do Postman
└── README.md
```

Para executar, são necessários dois terminais, um para cada aplicação:

```
# Clonar o repositório
git clone https://github.com/Wesleytech22/gerenciamento-pedidos.git
cd gerenciamento-pedidos

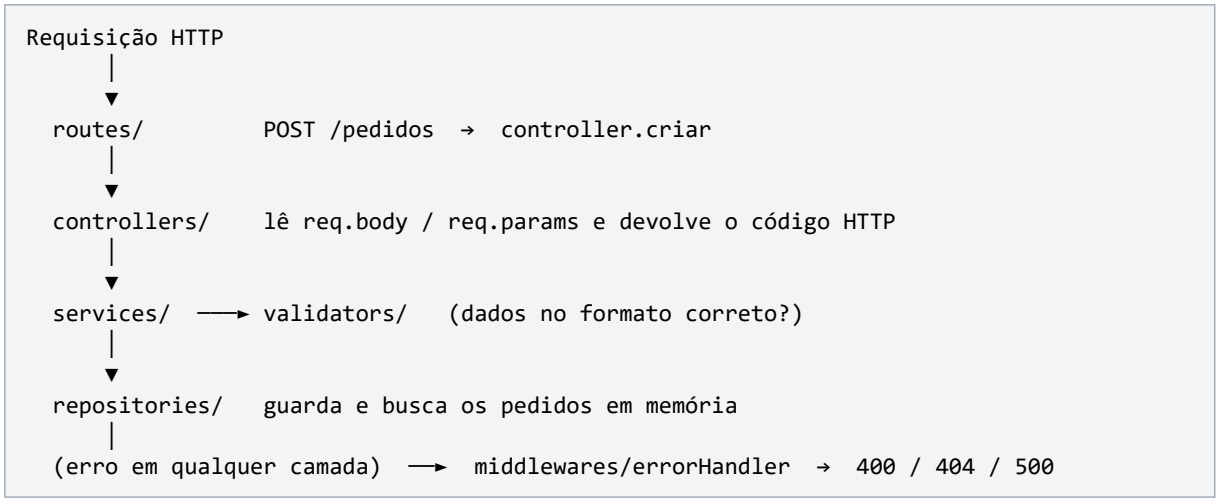
# Terminal 1 - backend
cd backend
npm install
npm start          # http://localhost:3000

# Terminal 2 - frontend (com o backend já rodando)
cd frontend
npm start          # http://localhost:5173

# Testes automatizados
cd backend
npm test
```

4 ARQUITETURA EM CAMADAS

A API segue uma arquitetura em camadas, em que cada requisição percorre o caminho **rota** → **controller** → **service** → **repository**. O padrão adotado é um MVC adaptado para API (sem a camada de visão), com camada de serviço, *Repository Pattern* e tratamento de erros centralizado em middleware.



Camada / pasta	Responsabilidade
`routes/`	Associa método HTTP e URL à função correspondente do controller.
`controllers/`	Traduz HTTP para o service: lê a requisição, chama o service e define o código de resposta (200, 201). Não contém regras de negócio.
`services/`	Regras de negócio: status inicial, verificação de existência do pedido e validação das transições de status.
`validators/`	Valida os campos recebidos e concentra as listas de valores aceitos (tipos, prioridades, status e transições).
`repositories/`	Armazena os pedidos em memória. O service não sabe onde os dados ficam; trocar por um banco de dados alteraria apenas esta camada. Retorna cópias dos objetos, para que nada altere os dados por fora.
`errors/`	Define ValidationError (400) e NotFoundError (404). O service sinaliza o tipo de erro sem conhecer detalhes do HTTP.
`middlewares/`	Converte os erros em respostas JSON: erros conhecidos viram 400/404 e qualquer erro inesperado vira 500, sem expor detalhes internos.

5 MODELO DE DADOS E VALORES ACEITOS

Cada pedido possui os campos abaixo. O **id** e o **status** são definidos pela API; os demais são enviados pelo cliente na criação.

Campo	Tipo	Regra
`id`	número inteiro	Gerado automaticamente, sequencial, a partir de 1.
`cliente`	texto	Obrigatório; não pode ser vazio nem conter apenas espaços. É salvo sem os espaços das extremidades (" Ana " vira "Ana").
`tipo`	texto	`entrega` ou `retirada`.
`valor`	número	Deve ser do tipo número no JSON e maior que zero. O texto "350" é recusado.
`prioridade`	texto	`baixa`, `normal` ou `alta`.

Campo	Tipo	Regra
`status`	texto	Sempre `recebido` na criação. Valores possíveis: `recebido`, `em_preparacao`, `enviado`, `entregue` e `cancelado`.

6 ROTAS DA API

Todas as rotas usam o endereço base **http://localhost:3000**, recebem e devolvem JSON. Nas requisições com corpo deve ser enviado o cabeçalho `Content-Type: application/json`.

Método	Rota	Descrição	Respostas
POST	`/pedidos`	Cadastra um pedido (RF01)	201 · 400
GET	`/pedidos`	Lista os pedidos (RF02)	200
GET	`/pedidos/:id`	Consulta um pedido (RF03)	200 · 404
PATCH	`/pedidos/:id/status`	Atualiza o status (RF04)	200 · 400 · 404

6.1 POST /pedidos — Cadastrar pedido

Cria um novo pedido. A API valida todos os campos, remove os espaços das extremidades do nome do cliente, gera o identificador e define o status inicial como `recebido`. Caso haja mais de um campo inválido, todos os erros são retornados juntos no campo `detalhes`.

Requisição:

```
POST http://localhost:3000/pedidos
Content-Type: application/json

{
  "cliente": "Carlos Silva",
  "tipo": "entrega",
  "valor": 350,
  "prioridade": "normal"
}
```

Resposta 201 Created:

```
{
  "id": 1,
  "cliente": "Carlos Silva",
  "tipo": "entrega",
  "valor": 350,
  "prioridade": "normal",
  "status": "recebido"
}
```

Exemplo de requisição inválida e resposta 400 Bad Request:

```
{ "cliente": "   ", "tipo": "drone", "valor": -5, "prioridade": "normal" }

{
  "erro": "Dados do pedido inválidos.",
  "detalhes": [
    "cliente é obrigatório e deve ser um texto não vazio.",
    "valor deve ser numérico e maior que zero.",
    "tipo deve ser um dos valores: entrega, retirada."
  ]
}
```

Situação	Código	Motivo
Todos os campos válidos	201	Pedido criado e devolvido com id e status.
Cliente ausente, vazio ou só com espaços	400	Cliente é obrigatório.
Valor zero, negativo ou enviado como texto	400	Valor deve ser numérico e maior que zero.
Tipo ou prioridade fora da lista	400	Só são aceitos os valores documentados.
Corpo com JSON malformado	400	"JSON da requisição está malformado."

6.2 GET /pedidos — Listar pedidos

Retorna todos os pedidos cadastrados, na ordem de criação. Quando não há pedidos, retorna uma lista vazia, também com código 200.

Requisição e resposta 200 OK:

```
GET http://localhost:3000/pedidos

[
  { "id": 1, "cliente": "Carlos Silva", "tipo": "entrega", "valor": 350,
    "prioridade": "normal", "status": "recebido" },
  { "id": 2, "cliente": "Maria Souza", "tipo": "retirada", "valor": 120.5,
    "prioridade": "alta", "status": "recebido" }
]
```

6.3 GET /pedidos/:id — Consultar pedido

Retorna o pedido cujo identificador foi informado na URL. Se o pedido não existir, ou se o identificador não for um número inteiro (por exemplo, `/pedidos/abc`), a API responde 404.

Pedido existente — 200 OK:

```
GET http://localhost:3000/pedidos/1

{ "id": 1, "cliente": "Carlos Silva", "tipo": "entrega", "valor": 350,
  "prioridade": "normal", "status": "recebido" }
```

Pedido inexistente — 404 Not Found:

```
GET http://localhost:3000/pedidos/999

{ "erro": "Pedido com id '999' não encontrado." }
```

6.4 PATCH /pedidos/:id/status — Atualizar status

Altera somente o status do pedido. A API verifica primeiro se o pedido existe (404) e depois se o status enviado é válido e se a transição é permitida (400). A ordem é importante: para um id inexistente a resposta é sempre 404, mesmo que o corpo também esteja inválido.

Requisição e resposta 200 OK:

```
PATCH http://localhost:3000/pedidos/1/status
Content-Type: application/json

{ "status": "em_preparacao" }

{ "id": 1, "cliente": "Carlos Silva", "tipo": "entrega", "valor": 350,
  "prioridade": "normal", "status": "em_preparacao" }
```

Transição não permitida — 400 Bad Request:

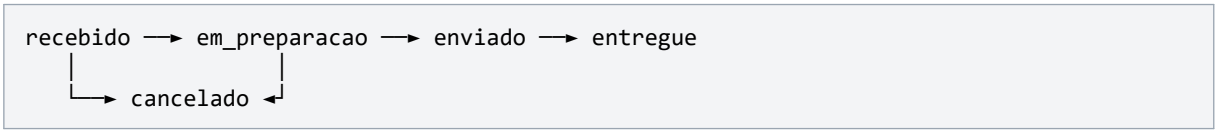
```
{ "status": "recebido" }

{ "erro": "Transição de 'em_preparacao' para 'recebido' não é permitida.
  Transições possíveis: enviado, cancelado." }
```

Situação	Código
Pedido existe e a transição é permitida	200
Status ausente no corpo ou fora da lista de status	400
Transição não permitida (ex.: recebido → entregue; alterar pedido cancelado)	400
Pedido inexistente	404

7 REGRAS DE TRANSIÇÃO DE STATUS

O status do pedido funciona como uma máquina de estados. O pedido nasce como `recebido` e segue o fluxo normal até `entregue`. O cancelamento só é possível antes do envio. Os status entregue e cancelado são finais.



Status atual	Pode ir para	Justificativa
`recebido`	`em_preparacao`, `cancelado`	Pedido ainda não começou a ser preparado.
`em_preparacao`	`enviado`, `cancelado`	Ainda está na loja, pode ser cancelado.

Status atual	Pode ir para	Justificativa
`enviado`	`entregue`	Já saiu da loja; não é mais possível cancelar.
`entregue`	— (final)	Pedido concluído.
`cancelado`	— (final)	Pedido encerrado.

8 TRATAMENTO DE ERROS

Todos os erros são devolvidos em JSON, com a mensagem no campo `erro` e, nas validações do cadastro, a lista de problemas no campo `detalhes`. O tratamento é centralizado no middleware errorHandler, que distingue erros do usuário de erros do servidor:

Origem	Código	Resposta
ValidationError (dados inválidos)	400	Mensagem explicativa e, quando houver, detalhes.
JSON malformado no corpo	400	"JSON da requisição está malformado."
NotFoundError (pedido inexistente)	404	"Pedido com id 'X' não encontrado."
Rota que não existe	404	"Rota GET /xyz não encontrada."
Qualquer erro inesperado	500	"Erro interno do servidor." O detalhe fica só no log do servidor.

Dessa forma, um erro inesperado nunca é apresentado como se fosse um problema de validação dos dados do usuário, conforme exigido no enunciado.

9 FRONTEND

O frontend é uma aplicação independente, na pasta frontend/, servida por um pequeno servidor estático em Node.js, sem dependências externas. Ele consome a API pelo endereço configurado em public/config.js e oferece:

- formulário para cadastrar pedidos (cliente, valor, tipo e prioridade);
- tabela com a lista de pedidos e o status de cada um;
- busca de um pedido pelo identificador;
- alteração do status, oferecendo apenas as transições permitidas para o status atual;
- exibição das mensagens de erro retornadas pela API e indicador de API online/offline.

Toda a validação continua sendo feita pela API: o frontend apenas exibe as mensagens que ela devolve. Assim, as regras de negócio ficam em um único lugar, o backend.

10 TESTES

10.1 Testes automatizados

Os testes ficam em backend/tests/ e são executados com `npm test`. São 33 casos, todos passando, divididos em testes de integração das rotas (Supertest) e testes de unidade do service:

- criação válida, com status inicial recebido e remoção de espaços do cliente;
- criação inválida: cliente ausente, vazio ou não texto; valor zero, negativo ou como texto; tipo e prioridade desconhecidos; corpo que não é objeto; JSON malformado;
- listagem vazia e com pedidos; consulta de pedido existente, inexistente e com id não numérico;
- fluxo completo de status até entregue, status desconhecido, transição proibida, alteração de pedido cancelado e 404 no PATCH;
- rota inexistente (404) e erro inesperado retornando 500 sem expor detalhes.

10.2 Testes manuais no Postman

A coleção backend/postman/API_Pedidos.postman_collection.json contém as requisições abaixo, que devem ser executadas nesta ordem (o pedido 1 só existe após o primeiro cadastro):

Nº	Requisição	Corpo	Esperado	Evidência
1	`POST /pedidos`	pedido válido	201	01-criacao.png
2	`POST /pedidos`	cliente vazio, tipo e valor inválidos	400	02-validacao.png
3	`GET /pedidos`	—	200	03-listagem.png
4	`GET /pedidos/999`	—	404	04-consulta-inexistente.png
5	`PATCH /pedidos/1/status`	`{"status":"em_preparacao"}`	200	05-atualizacao.png
6	`GET /pedidos/1`	—	200	—
7	`PATCH /pedidos/1/status`	`{"status":"recebido"}`	400	—

As capturas das requisições 1 a 5 estão na pasta evidencias/ do repositório e exibem o método, a URL, o corpo enviado, a resposta e o código HTTP.

11 CONSIDERAÇÕES FINAIS

A API atende aos quatro requisitos funcionais e aos códigos HTTP exigidos. A divisão em camadas deixa claro onde cada responsabilidade está: as rotas apenas direcionam, os controllers tratam o HTTP, os services aplicam as regras de negócio, os validators conferem os dados e o repositório guarda as informações. Essa organização facilita os testes, a localização de erros e uma futura troca do armazenamento em memória por um banco de dados.